

هومرب: داده‌ها از کجا آمده‌اند و بک‌اند چطور کار می‌کند

گزارش فنی برای محمدجواد · وضعیت کد روی شاخهٔ main · مهر ۱۴۰۵ (سپتامبر ۲۰۲۶)

جواب کوتاه به سه سؤال

از کجا آمده؟

همهٔ ۱۰۰ آگهی ساختگی هستند. یک اسکریپت Node به اسم `scripts/generate-listings.mjs` آن‌ها را با اعداد تصادفی قابل تکرار ساخته و در `src/data/listings.json` ذخیره کرده است. هیچ آگهی واقعی‌ای در پروژه نیست.

دیوار یا شیپور؟

هیچ‌کدام. هیچ چیزی از دیوار یا شیپور خوانده، کپی یا اسکرپ نشده و هیچ API‌ای از آن‌ها صدا زده نمی‌شود. فیلد `source` (دیوار/شیپور) هم برای هر آگهی با قرعهٔ تصادفی تعیین شده (۶۰٪ دیوار، ۴۰٪ شیپور) تا حس «متاسرچ چند منبعی» در دمو دیده شود.

Hugging Face؟

نه. نه دیتاست و نه مدلی از Hugging Face استفاده نشده. تنها هوش مصنوعی پروژه Google Gemini (سطح رایگان) است که از طریق API صدا زده می‌شود و فقط کوئری کاربر را می‌فهمد و توضیح می‌نویسد. داده نمی‌سازد.

این تصمیم‌ها از قبل در `docs/DECISIONS.md` ثبت شده‌اند: «داده: *seeded sample dataset*، فایل JSON، بدون دیتابیس، بدون اسکرپ زنده». اسکرپ دیوار و شیپور جزو «خط قرمزها»ی `CLAUDE.md` هم هست. دلیلش سه چیز است: قوانین استفادهٔ این سایت‌ها اجازهٔ اسکرپ نمی‌دهد، مهلت دو روزه بود، و دمو باید همیشه یک خروجی ثابت و قابل تکرار بدهد.

۱. داده‌ها دقیقاً چطور ساخته شدند

مولد قطعی (deterministic)

اسکریپت از یک مولد عدد تصادفی بذردار (الگوریتم `mulberry32` با بذر `20260924`) استفاده می‌کند. نتیجه این است که `npm run generate:listings` هر بار دقیقاً همان ۱۰۰ آگهی را می‌سازد. ترکیب داده این‌طور است:

- ۹۰ آگهی خودکار: ۶ محله × ۱۵ آگهی.
- ۱۰ آگهی دست‌ساز برای سناریوهای دمو و کیفیت داده (جزئیات پایین‌تر).
- توزیع نهایی منبع: ۵۷ دیوار و ۴۳ شیپور. توزیع محله‌ها: وکیل آباد ۲۱، هاشمیه ۱۷، سجاد ۱۶، قاسم‌آباد ۱۶، احمدآباد ۱۵، الهیه ۱۵.

مدل قیمت‌گذاری

برای هر محله یک بازهٔ «قیمت هر متر به صورت رهن کامل» تعریف شده است. این بازه‌ها تخمین من از سطح بازار مشهد در ۲۰۲۶ هستند و با داده‌های واقعی اعتبارسنجی نشده‌اند:

محله	قیمت هر متر (رهن کامل، تومان)	احتمال «نزدیک قطار شهری»	سهم ساختمان نوساز
الهیة	۱۷ تا ۲۲ میلیون	۱۰٪	۵۰٪
سجاد	۱۴ تا ۱۸ میلیون	۳۵٪	۳۰٪
احمدآباد	۱۳ تا ۱۷ میلیون	۴۰٪	۱۵٪
وکیل آباد	۱۲ تا ۱۶ میلیون	۵۵٪	۳۵٪
هاشمیه	۱۱ تا ۱۴ میلیون	۱۵٪	۳۵٪
قاسم آباد	۸ تا ۱۱ میلیون	۴۵٪	۲۰٪

مراحل ساخت هر آگهی:

- مشخصات:** تعداد خواب با وزن مخصوص هر محله انتخاب می‌شود. متراژ به تعداد خواب بستگی دارد (سوئیت ۳۵ تا ۵۰ متر، دوخوابه ۸۰ تا ۱۲۵ متر و ...). بعد سن بنا، طبقه، آسانسور، پارکینگ، انباری و برجسب‌ها (بالکن، مبله، لابی‌من، ...) تعیین می‌شوند.
- قیمت کل:** قیمت متری از بازه محله گرفته و اصلاح می‌شود: نوساز ۱۲٪+، بالای ۱۹ سال ۲۰٪-، پارکینگ ۴٪+، آسانسور ۳٪+، مبله ۱۰٪+. در آخر $\pm 7\%$ نویز اضافه می‌شود تا بعضی آگهی‌ها ارزان و بعضی گران‌تر از عرف محله باشند. حاصل ضرب این عدد در متراژ، «معادل رهن کامل» آگهی است.
- تقسیم به رهن و اجاره:** ۱۵٪ آگهی‌ها رهن کامل‌اند، ۷۸٪ ترکیبی (۲۰ تا ۶۵ درصد به صورت رهن) و ۷٪ عمدتاً اجاره. باقی‌مانده با نرخ عرفی **ماهانه ۳٪** (هر ۱ میلیون رهن = ۳۰ هزار اجاره) به اجاره تبدیل و گرد می‌شود.
- متن:** عنوان و توضیحات از قالب‌های فارسی شبیه دیوار/شیپور ساخته می‌شوند، مثل «مالک هستم، مشاور نیستم.» یا «رهن و اجاره قابل جابجایی». تاریخ انتشار هم تصادفی بین ۰ تا ۲۱ روز قبل است.

۱۰ آگهی دست‌ساز و دلیل هرکدام

شناسه	نقش در دمو
dv-0901 ... dv-0904	دوخوابه‌های وکیل‌آباد حول بودجه ۵۰۰ میلیون رهن، برای کوئری اصلی دمو. هرکدام یک مصالحه متفاوت دارند: بدون پارکینگ، بدون آسانسور، نوساز ولی گران‌تر.
sp-0905	همان واحد dv-0901 که در شیپور هم آگهی شده. این آگهی برای نمایش ادغام تکراری‌ها (مثل «در شیپور هم هست» در ترب) است.
dv-0906	رهن کامل ۱/۱ میلیاردی در هاشمیه، برای مقایسه رهن کامل با ترکیبی.
dv-0907, sp-0908	قیمت‌های بی‌معنی («توافقی»، ۱۰۰۰ تومان، ۱۱۱۱۱۱۱). سیستم باید این‌ها را تشخیص دهد و در رتبه‌بندی به‌عنوان «ارزان» حساب نکند.
dv-0909, sp-0910	اجاره اتاق/همخونه. قیمتشان برای یک اتاق است، پس فقط وقتی نمایش داده می‌شوند که کاربر خودش همخونه بخواهد.

```
{
  "id": "sp-0902",
  "source": "sheypoor",
  "title": "رهن و اجاره آپارتمان ۱۰۵ متری دوخوابه در وکیل‌آباد",
  "neighborhood": "وکیل‌آباد", "street": "وکیل‌آباد",
  "deposit": 550000000, "monthlyRent": 160000000, // Toman
  "areaM2": 105, "rooms": 2, "floor": 4, "totalFloors": 6, "buildingAge": 3,
  "elevator": true, "parking": true, "storage": true, "convertible": true,
  "tags": ["بالکن", "پکیج", "نزدیک قطار شهری"],
  "description": "... طبقه ۴ از ۶، ۳ سال ساخت",
  "postedAt": "2026-09-24T07:10:00.000Z"
}
```

نوع کامل در `src/lib/types.ts` است. آگهی‌ها عکس ندارند. مختصات روی نقشه هم ساختگی است: تابع `listingLatLng` در `src/lib/geo.ts` از روی شناسه آگهی یک نقطه ثابت داخل محله می‌سازد.

داده خارجی واقعی که استفاده شده

در کل پروژه فقط این منابع خارجی به کار رفته‌اند و هیچ‌کدام آگهی مسکن نیستند:

منبع	کاربرد	مجوز / نحوه
geoBoundaries (مرز استان‌های ایران)	انیمیشن نقشه صفحه اول	CC BY 4.0، یک‌بار دانلود و با <code>scripts/build-hero-map.mjs</code> به SVG تبدیل شده
OpenStreetMap از طریق Overpass (خیابان‌های اصلی مشهد و مرکز محله‌ها)	نقشه صفحه اول و جای محله‌ها	ODbL، یک‌بار دانلود، در کد ذخیره شده
نقشه نشان (Neshan Web SDK)	تایل‌های نقشه نتایج	کلید رایگان وب، محدود به دامنه، در <code>env</code> و رسل
هدرهای IP و رسل	شهر تقریبی بازدیدکننده وقتی اجازه موقعیت نمی‌دهد	بدون سرویس ثالث
Google Gemini API	فهم کوئری و نوشتن توضیحات	سطح رایگان، کلید فقط در <code>env</code>

نکته برای ویدیو: بهتر است در ویدیو صریح بگویید «داده‌ها نمونه و ساختگی‌اند، به سبک دیوار و شیپور». README هم همین را می‌گوید. اگر دآوری بپرسد، جواب صادقانه و محکم این است: لایه داده جدا شده و قابل تعویض است و هسته محصول لایه هوش مصنوعی و رتبه‌بندی است. (یک اصلاح کوچک: README هنوز می‌نویسد ۹۶ آگهی، ولی عدد فعلی ۱۰۰ است.)

اگر بخواهیم داده واقعی بیاوریم (کار آینده)

- **API رسمی:** مثلاً پلتفرم «کنار» دیوار برای شریک‌ها. این راه قانونی است ولی نیاز به حساب و مجوز دارد.

- منبع فقط خواندنی مثل یک سرور MCP. در DECISIONS.md ثبت شده که از divar-mcp فقط ایده گرفته ایم و هیچ اتصالی به آن نداریم.
 - دیتاست عمومی (مثلاً روی Hugging Face یا Kaggle)، به شرط اینکه مجوزش اجازه بدهد. فقط برای کالیبره کردن قیمت‌های محله‌ها هم مفید است. من چنین دیتاستی را بررسی یا استفاده نکرده‌ام.
- در هر سه حالت فقط باید src/data/listings.ts عوض شود و خروجی جدید را به شکل Listing[] بدهد. بقیه سیستم دست نمی‌خورد.

۲. معماری بک‌اند

بک‌اند جدایی وجود ندارد. کل پروژه یک اپ Next.js 16 (TypeScript) است که روی Vercel اجرا می‌شود. «بک‌اند» همان API Routes است که روی Vercel به شکل توابع serverless اجرا می‌شوند. دیتابیس نداریم: فایل JSON آگهی‌ها هنگام build داخل باندل قرار می‌گیرد و جست‌وجو روی آرایه‌ای در حافظه انجام می‌شود. با ۱۰۰ آگهی این کار چند میلی‌ثانیه طول می‌کشد.

Endpoint	ورودی	خروجی و کار
POST /api/search	query (متن فارسی، ۲ تا ۳۰۰ کاراکتر)، intent (اختیاری، وقتی کاربر چیزی را حذف کرده)، near (محله کاربر، اختیاری)	نیت کاربر را می‌فهمد، فیلتر و رتبه‌بندی می‌کند و همه نتایج جاشده در بودجه را با امتیاز، نقاط قوت و ضعف و توضیح قاعده‌محور برمی‌گرداند. وقتی نتیجه صفر است یک پیشنهاد هم می‌دهد. meta نشان می‌دهد نیت از AI آمده یا از قواعد.
POST /api/explain	query ، intent ، و تا ۱۰ شناسه آگهی	یک درخواست دسته‌ای به LLM می‌فرستد و برای هر آگهی ۱ تا ۲ جمله فارسی «چرا به دردت می‌خورد» می‌گیرد. این endpoint جدا است تا نتایج بدون معطلی نمایش داده شوند و توضیحات بعداً جایگزین شوند.
GET /api/geo	—	کشور، شهر و مختصات تقریبی را از هدرهای IP و رسل می‌خواند (برای انیمیشن نقشه).

ورودی همه endpoint ها با zod اعتبارسنجی می‌شود و ورودی نامعتبر خطای 400 می‌گیرد.

مسیر یک جست‌وجو از اول تا آخر

۱ فهم کوئری (Intent parsing): `src/lib/intent/`

متن فارسی به یک شیء ساختاریافته تبدیل می‌شود: سقف رهن، سقف اجاره، محله‌ها، حداقل و حداکثر خواب، حداقل مترآز، امکانات ضروری و ترجیحی، همخونه، یادداشت. این کار با Gemini و یک پرامپت few-shot انجام می‌شود.

۲ ادغام تکراری‌ها: `search/dedup.ts`

یک واحد که هم در دیوار و هم در شیپور آمده، یک کارت می‌شود با برچسب «در شیپور هم هست».

۳ فیلتر کیفیت: `lib/quality.ts`

قیمت‌های بی‌معنی و آگهی‌های همخونه کنار گذاشته و شمرده می‌شوند. AI این تعداد را نشان می‌دهد (مثلاً «۲ آگهی با قیمت توافقی نمایش داده نشد»).

۴ فیلتر سخت بودجه: `search/budget.ts`

آگهی بالاتر از بودجه حذف می‌شود. اگر صاحب‌خانه و کاربر هر دو قابل‌تبدیل بودن را قبول کنند، رهن و اجاره با نرخ ۳٪ جابه‌جا می‌شوند تا ببینیم با تبدیل‌جا می‌شود یا نه.

۵ امتیاز نرم و رتبه‌بندی: `search/score.ts`

امتیاز ۰ تا ۱۰۰ از هفت معیار وزن‌دار. همین‌جا نقاط قوت و ضعف و یک توضیح قاعده‌محور هم ساخته می‌شود.

۶ توضیح هوش مصنوعی: `lib/explain/`

برای ۱۰ نتیجه اول یک درخواست دسته‌ای فرستاده می‌شود. هر عددی که LLM بنویسد ولی در داده نباشد رد می‌شود و به‌جایش توضیح قاعده‌محور می‌نشیند.

۳. جزئیات هر لایه

الف) کلاینت هوش مصنوعی: `src/lib/ai/client.ts`

- یک کلاینت کوچک و سازگار با OpenAI که بدون SDK و فقط با `fetch` به `/chat/completions` درخواست می‌فرستد. Gemini، DeepSeek و OpenAI همه همین پروتکل را می‌فهمند، پس برای عوض کردن provider کافی است متغیر `AI_PROVIDER` و کلیدش را عوض کنیم.
- روی Production، provider برابر `gemini` است و این سه مدل به ترتیب امتحان می‌شوند: `gemini-3.5-flash-lite` → `gemini-flash-lite-latest` → `gemini-3.6-flash`.

- **درخواست موازی پشتیبان (hedging):** اگر مدل اول بعد از ۱/۵ ثانیه جواب نداد، مدل دوم هم موازی شروع می‌شود و اولین جواب معتبر برنده است. سقف کل ۵ ثانیه است (برای توضیح‌ها ۳ و ۷ ثانیه). خطاهای 429، 5xx و 404 باعث رفتن به مدل بعدی می‌شوند و بقیه 4xx ها نه.
- از LLM خروجی `json_object` خواسته می‌شود و پاسخ حتماً باید JSON معتبر باشد.
- اگر کلید تنظیم نشده باشد، provider خودبه‌خود `mock` می‌شود و همه‌چیز با قواعد کار می‌کند.

ب) فهم کوئری: `src/lib/intent/`

- `llm.ts`: پرامپت انگلیسی با قواعد پول به زبان محاوره‌ای. مثلاً «ماهی ۸ تومن» یعنی ۸ میلیون، «رهن ۱/۵» یعنی ۱/۵ میلیارد و «پونصد» یعنی ۵۰۰. دو مثال `few-shot` دارد و `temperature` آن صفر است. قبل از اعتبارسنجی سخت با `SearchIntentSchema`، یک مرحله پاکسازی ملایم اجرا می‌شود: نام محله‌ها استاندارد می‌شود، عدد داخل رشته به عدد تبدیل می‌شود و امکانات ناشناخته حذف می‌شوند.
- `rules.ts`: پارسر قطعی فارسی با `regex`. هم حالت `mock` است و هم وقتی LLM شکست بخورد جایگزین می‌شود. برای همین جست‌وجو هیچ‌وقت از کار نمی‌افتد.
- **کش:** یک Map در حافظه برای ۳۰۰ کوئری آخر، با کلید متن نرمال‌شده. جواب‌های `fallback` کش نمی‌شوند تا دفعه بعد دوباره از LLM پرسیده شود.
- **«نزدیک من»:** فیلد `nearMe` را LLM پر نمی‌کند. سرور آن را از مکان مرورگر کاربر می‌گذارد، آن هم فقط وقتی که کوئری اسم محله‌ای نیاورده باشد.

ج) منطق بودجه: `src/lib/search/budget.ts`

پایه همه مقایسه‌ها «معادل رهن کامل» است: $fullDeposit = deposit + monthlyRent / 0.03$.

حالت	قاعده
هر دو سقف گفته شده	جا می‌شود اگر رهن $\geq$ سقف رهن و اجاره $\geq$ سقف اجاره باشد، یا اگر با تبدیل (فقط وقتی آگهی قابل‌تبدیل است) بشود این دو شرط را برقرار کرد.
فقط سقف رهن گفته شده	تا ۳۰٪ بالاتر از سقف با تبدیل پذیرفته می‌شود (<code>ONE_SIDED_STRETCH</code>). اجاره هم سقف ضمنی دارد: ۲ برابر معادل اجاره بودجه (<code>IMPLICIT_FACTOR</code>). بدون این سقف هر آگهی‌ای با اجاره خیلی بالا «جا می‌شد».
فقط سقف اجاره گفته شده	همان منطق برعکس. «ماهی ۸ تومن» آگهی رهن کامل ۱/۴ میلیاردی را نشان نمی‌دهد.

مثال با کوئری دمو «دوخواه وکیل‌آباد با ۵۰۰ میلیون رهن»:

- `dv-0901`: رهن ۴۵۰ و اجاره ۱۸. رهن زیر سقف است و اجاره زیر سقف ضمنی ($2 \times 500 \times 3\% = 30$ میلیون)، پس جا می‌شود و «۵۰۰ میلیون زیر بودجه» است. پارکینگ ندارد و همین در نقاط ضعفش می‌آید.
- `sp-0902`: رهن ۵۵۰ و اجاره ۱۶، قابل‌تبدیل. ۵۵۰ کمتر از $650 \times (1/3 \times 500)$ است، پس با تبدیل به رهن ۵۰۰ اجاره می‌شود $16 + 50 \times 3\% = 17.5$ میلیون. جا می‌شود و کارت نشان می‌دهد «با تبدیل».

د) امتیازدهی: `src/lib/search/score.ts`

معیار	وزن	نحوه محاسبه
محله	۳۰	محله خواسته شده ۱، محله همسایه ۰/۴۵، بقیه ۰. در حالت «نزدیک من» محله خود کاربر ۱ و همسایه ها ۰/۶.
امکانات ضروری	۲۵	سهم امکانات ضروری موجود در آگهی.
تعداد خواب	۲۰	در بازه ۱. یک خواب کمتر ۰/۳. خیلی بزرگتر از خواسته ۰/۷۵.
متراژ	۸	کمبود نسبت به حداقل متراژ با شیب تند جریمه می شود.
ارزش خرید	۷	قیمت متری آگهی در مقایسه با میانۀ محله : ۲۰٪ ارزان تر ۱، برابر میانه ۰/۵، ۲۰٪ گران تر ۰.
امکانات ترجیحی	۵	سهم امکانات ترجیحی موجود.
تازگی	۵	خطی از ۱ برای جدیدترین آگهی تا ۰ برای ۲۱ روز قبل.

جمع وزن ها ۱۰۰ است، پس امتیاز همان درصد تطابق است. با امتیاز برابر، آگهی ارزان تر (بر اساس معادل رهن کامل) جلوتر می آید. میانه هر محله فقط از آگهی های «قابل مقایسه» محاسبه می شود (بدون قیمت های بی معنی و همخونه) و الی تعداد نمونه را هم کنارش نشان می دهد.

ه) توضیح هوش مصنوعی: `src/lib/explain/`

- فقط واقعیت های محاسبه شده (نقاط قوت و ضعف و اعداد) به LLM داده می شوند و در پرامپت گفته شده «چیزی از خودت نسا».
- گارد عدد (`grounding.ts`): هر عددی در متن LLM که در داده های آن آگهی نباشد باعث رد شدن متن می شود و توضیح قاعده محور جایش را می گیرد. این جلوی قیمت سازی خیالی را می گیرد.
- امنیت: کلاینت فقط شناسه آگهی ها را می فرستد و سرور واقعیت ها را خودش دوباره محاسبه می کند (`resultsByIds`). پس نمی شود با درخواست جعلی LLM را به گفتن چیز غلط واداشت.
- نتیجه کش می شود و اگر درخواست شکست بخورد یا به محدودیت نرخ (429) بخورد، توضیح قاعده محور می ماند.

و) کارهایی که عمداً سمت کلاینت است

`/api/search` همه نتایجی را که در بودجه جا می شوند برمی گرداند. فیلترهای بعدی (بازه قیمت و متراژ، هیستوگرام، محله، امکانات، مرتب سازی، جست و جو در این محدوده» روی نقشه) با توابع خالص در `src/lib/search/refine.ts` روی مرورگر اجرا می شوند. به همین دلیل هر تغییر فیلتر بدون رفت و برگشت به سرور، فوری اعمال می شود. حذف یک چیپ اما دوباره به سرور می رود، این بار با `intent` ویرایش شده و بدون صدا زدن LLM.

۴. کارایی، محدودیت‌ها و ریسک‌ها

محدودیت‌ها

- داده ساختگی است و قیمت‌ها تخمینی و اعتبارسنجی نشده‌اند.
- سطح رایگان Gemini گاهی کند است یا 429 و 503 می‌دهد. در آن حالت پارسر قواعد جایش را می‌گیرد (meta.intentSource = "rules").
- کش در حافظه هر نمونه serverless است و بین نمونه‌ها یا بعد از cold start مشترک نیست.
- تشخیص تکراری فقط با تطابق دقیق کار می‌کند.

اعداد اندازه‌گیری‌شده روی Production

- جست‌وجو: میانه حدود ۱/۷ ثانیه، حداکثر حدود ۵ ثانیه (سقف طراحی‌شده).
- کوئری تکراری: حدود ۱۵۰ میلی‌ثانیه (از کش).
- توضیح‌ها: ۲ تا ۷ ثانیه، به صورت async و بعد از نمایش نتایج.
- ۵۹ تست واحد با vitest (بودجه، قیمت، کیفیت، پارسر، رتبه‌بندی، گارد عدد).

نقشه فایل‌ها (بخش سرور)

scripts/generate-listings.mjs	← generates the 100 sample listings (seeded)
scripts/build-hero-map.mjs	← OSM + geoBoundaries → SVG for the home map
src/data/listings.json .ts	← the dataset (the only thing to swap for real data)
src/app/api/search/route.ts	← POST /api/search
src/app/api/explain/route.ts	← POST /api/explain
src/app/api/geo/route.ts	← GET /api/geo
src/lib/ai/client.ts	← OpenAI-compatible client, hedged model race
src/lib/intent/{llm,rules,schema,index}.ts	← query → SearchIntent
src/lib/search/{index,budget,score,dedup,suggest,refine}.ts	
src/lib/explain/{index,grounding}.ts	← batched AI explanations + number guard
src/lib/{pricing,quality,geo,neighborhoods}.ts	

این گزارش از روی کد مخزن MohammadJavadHeidari/Homerob نوشته شده است (commit b6a623d). جزئیات تصمیم‌ها در docs/PLAN.md و وضعیت کارها در docs/DECISIONS.md است.